Myntra Automation Framework

TestNG + Selenium + Allure | Page Object Model

Folder Structure

```
automation-framework/
├── .gitignore
├── pom.xml
├── testng.xml
├── allure.properties
│
├── src/main/java/com/company/project/
│   ├── pages/
│   │   ├── BasePage.java
│   │   ├── LoginPage.java
│   │   ├── HomePage.java
│   │   ├── SearchPage.java
│   │   ├── ProductPage.java
│   │   └── CartPage.java
│   ├── drivers/
│   │   ├── DriverFactory.java
│   │   └── DriverManager.java
│   ├── listeners/
│   │   ├── TestListener.java
│   │   ├── RetryAnalyzer.java
│   │   └── RetryListener.java
│   └── utils/
│       ├── ConfigReader.java
│       ├── WaitUtils.java
│       └── ExcelUtils.java
│
├── src/test/java/com/company/project/
│   ├── tests/
│   │   ├── BaseTest.java
│   │   ├── LoginTest.java
│   │   ├── SearchTest.java
│   │   ├── ProductTest.java
│   │   └── CartTest.java
│   └── runners/
│       └── TestRunner.java
│
├── src/test/resources/
│   ├── config/
```

```
│   │   └── config.properties
│   ├── testdata/
│   │   ├── LoginTestData.xlsx
│   │   ├── SearchTestData.xlsx
│   │   ├── ProductTestData.xlsx
│   │   └── CartTestData.xlsx
│   └── suites/
│       ├── smoke-suite.xml
│       ├── regression-suite.xml
│       └── sanity-suite.xml
│
├── allure-results/
├── allure-report/
└── reports/
    ├── screenshots/
    └── logs/
```

File Descriptions

File	What it does
`BasePage.java`	Common methods all page objects inherit — click, type, wait
`LoginPage.java`	Locators and actions for login screen
`SearchPage.java`	Locators and actions for search bar and results
`ProductPage.java`	Locators and actions for product detail page
`CartPage.java`	Locators and actions for cart — quantity, unit price, total price
`BaseTest.java`	`@BeforeMethod` opens browser, `@AfterMethod` quits browser
`TestRunner.java`	TestNG runner pointing to feature files and glue code
`config.properties`	Browser, base URL, timeouts, retry count — changes per environment
`testng.xml`	Default suite file wiring listeners and test classes
`allure.properties`	Points Allure to results folder

Core Files — Full Code

config.properties

```properties
browser          = chrome
base.url         = https://www.myntra.com
implicit.wait    = 10
explicit.wait    = 20
page.load.timeout = 30
max.retry.count  = 2
```

DriverManager.java

```java
package com.company.project.drivers;

import org.openqa.selenium.WebDriver;

public class DriverManager {

    // Each thread gets its own WebDriver — safe for parallel runs
    private static ThreadLocal<WebDriver> driverThread = new ThreadLocal<>();

    public static void setDriver(WebDriver driver) {
        driverThread.set(driver);
    }

    public static WebDriver getDriver() {
        return driverThread.get();
    }

    public static void quitDriver() {
        if (driverThread.get() != null) {
            driverThread.get().quit();
            driverThread.remove();
        }
    }
}
```

DriverFactory.java

```
java
```

```java
package com.company.project.drivers;

import io.github.bonigarcia.wdm.WebDriverManager;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.firefox.FirefoxOptions;
import org.openqa.selenium.edge.EdgeDriver;
import org.openqa.selenium.edge.EdgeOptions;

public class DriverFactory {

    public static void initDriver(String browser) {
        WebDriver driver;

        switch (browser.toLowerCase().trim()) {

            case "chrome":
                WebDriverManager.chromedriver().setup();
                ChromeOptions chrome = new ChromeOptions();
                chrome.addArguments("--start-maximized");
                chrome.addArguments("--disable-notifications");
                driver = new ChromeDriver(chrome);
                break;

            case "firefox":
                WebDriverManager.firefoxdriver().setup();
                FirefoxOptions firefox = new FirefoxOptions();
                firefox.addArguments("--start-maximized");
                driver = new FirefoxDriver(firefox);
                break;

            case "edge":
                WebDriverManager.edgedriver().setup();
                EdgeOptions edge = new EdgeOptions();
                edge.addArguments("--start-maximized");
                driver = new EdgeDriver(edge);
                break;

            default:
                throw new IllegalArgumentException(
                    "Browser not supported: " + browser
                );
        }
```

```
        DriverManager.setDriver(driver);
    }
}
```

TestListener.java

```
java
```

```java
package com.company.project.listeners;

import com.company.project.drivers.DriverManager;
import io.qameta.allure.Allure;
import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;
import org.testng.ITestListener;
import org.testng.ITestResult;
import java.io.ByteArrayInputStream;
import java.io.File;
import java.io.FileOutputStream;
import java.text.SimpleDateFormat;
import java.util.Date;

public class TestListener implements ITestListener {

    @Override
    public void onTestStart(ITestResult result) {
        System.out.println("===== STARTED  : " + result.getName() + " =====");
    }

    @Override
    public void onTestSuccess(ITestResult result) {
        System.out.println("===== PASSED   : " + result.getName() + " =====");
    }

    @Override
    public void onTestFailure(ITestResult result) {
        System.out.println("===== FAILED   : " + result.getName() + " =====");
        System.out.println("===== REASON   : " + result.getThrowable().getMessage());

        byte[] screenshot = ((TakesScreenshot) DriverManager.getDriver())
                            .getScreenshotAs(OutputType.BYTES);

        // Save to local reports/screenshots folder
        try {
            String timestamp = new SimpleDateFormat("yyyyMMdd_HHmmss").format(new Date());
            String path = "reports/screenshots/" + result.getName() + "_" + timestamp + ".png";
            FileOutputStream fos = new FileOutputStream(new File(path));
            fos.write(screenshot);
            fos.close();
        } catch (Exception e) {
            e.printStackTrace();
        }

        // Attach to Allure report
```

```
        Allure.addAttachment(
            "Screenshot - " + result.getName(),
            new ByteArrayInputStream(screenshot)
        );
    }

    @Override
    public void onTestSkipped(ITestResult result) {
        System.out.println("===== SKIPPED  : " + result.getName() + " =====");
        Allure.step("Skipped: " + result.getName());
    }
}
```

RetryAnalyzer.java

```
java

package com.company.project.listeners;

import com.company.project.utils.ConfigReader;
import org.testng.IRetryAnalyzer;
import org.testng.ITestResult;

public class RetryAnalyzer implements IRetryAnalyzer {

    private int retryCount = 0;
    private static final int MAX_RETRY =
        Integer.parseInt(ConfigReader.get("max.retry.count"));

    @Override
    public boolean retry(ITestResult result) {
        if (retryCount < MAX_RETRY) {
            retryCount++;
            System.out.println("===== RETRYING : " + result.getName()
                + " | Attempt " + retryCount + " of " + MAX_RETRY + " =====");
            return true;
        }
        System.out.println("===== GIVING UP: " + result.getName()
            + " failed after " + MAX_RETRY + " retries =====");
        return false;
    }
}
```

RetryListener.java

```java
package com.company.project.listeners;

import org.testng.IAnnotationTransformer;
import org.testng.annotations.ITestAnnotation;
import java.lang.reflect.Constructor;
import java.lang.reflect.Method;

public class RetryListener implements IAnnotationTransformer {

    @Override
    public void transform(ITestAnnotation annotation,
                          Class testClass,
                          Constructor testConstructor,
                          Method testMethod) {
        // Auto applies RetryAnalyzer to every @Test — no manual wiring needed
        annotation.setRetryAnalyzer(RetryAnalyzer.class);
    }
}
```

ConfigReader.java

```java
```

```java
package com.company.project.utils;

import java.io.FileInputStream;
import java.util.Properties;

public class ConfigReader {

    private static Properties properties = new Properties();

    static {
        try {
            FileInputStream fis = new FileInputStream(
                "src/test/resources/config/config.properties"
            );
            properties.load(fis);
        } catch (Exception e) {
            throw new RuntimeException("config.properties not found: " + e.getMessage());
        }
    }

    public static String get(String key) {
        String value = properties.getProperty(key);
        if (value == null) throw new RuntimeException("Key not found: " + key);
        return value.trim();
    }
}
```

WaitUtils.java

```java
```

```java
package com.company.project.utils;

import com.company.project.drivers.DriverManager;
import org.openqa.selenium.By;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;
import java.time.Duration;

public class WaitUtils {

    private static int WAIT = Integer.parseInt(ConfigReader.get("explicit.wait"));

    public static WebElement waitForVisible(By locator) {
        return new WebDriverWait(DriverManager.getDriver(), Duration.ofSeconds(WAIT))
                .until(ExpectedConditions.visibilityOfElementLocated(locator));
    }

    public static WebElement waitForClickable(By locator) {
        return new WebDriverWait(DriverManager.getDriver(), Duration.ofSeconds(WAIT))
                .until(ExpectedConditions.elementToBeClickable(locator));
    }

    public static void waitForPriceToStabilize(By priceLocator) {
        // Wait for loader to disappear then wait for price to be visible
        new WebDriverWait(DriverManager.getDriver(), Duration.ofSeconds(WAIT))
                .until(ExpectedConditions.invisibilityOfElementLocated(
                    By.cssSelector(".loader, .spinner")
                ));
        waitForVisible(priceLocator);
    }
}
```

ExcelUtils.java

```java
```

```java
package com.company.project.utils;

import com.company.project.constants.ExcelConstants;
import org.apache.poi.ss.usermodel.*;
import org.apache.poi.xssf.usermodel.XSSFWorkbook;
import org.testng.annotations.DataProvider;
import java.io.FileInputStream;

public class ExcelUtils {

    // Core reader — reads any sheet from any file
    public static Object[][] readExcel(String filePath, String sheetName) throws Exception {
        FileInputStream fis = new FileInputStream(filePath);
        Workbook workbook   = new XSSFWorkbook(fis);
        Sheet sheet         = workbook.getSheet(sheetName);

        int rowCount        = sheet.getPhysicalNumberOfRows();
        int colCount        = sheet.getRow(0).getPhysicalNumberOfCells();
        Object[][] data     = new Object[rowCount - 1][colCount];

        for (int i = 1; i < rowCount; i++) {
            Row row = sheet.getRow(i);
            for (int j = 0; j < colCount; j++) {
                data[i - 1][j] = getCellValue(row.getCell(j));
            }
        }

        workbook.close();
        fis.close();
        return data;
    }

    private static Object getCellValue(Cell cell) {
        if (cell == null) return "";
        switch (cell.getCellType()) {
            case STRING:  return cell.getStringCellValue();
            case NUMERIC: return String.valueOf((int) cell.getNumericCellValue());
            case BOOLEAN: return cell.getBooleanCellValue();
            default:      return "";
        }
    }

    // Data Providers — one per module
    @DataProvider(name = "loginData")
    public static Object[][] getLoginData() throws Exception {
        return readExcel(ExcelConstants.LOGIN_DATA_PATH, ExcelConstants.LOGIN_SHEET);
```

```
    }

    @DataProvider(name = "searchData")
    public static Object[][] getSearchData() throws Exception {
        return readExcel(ExcelConstants.SEARCH_DATA_PATH, ExcelConstants.SEARCH_SHEET);
    }

    @DataProvider(name = "cartData")
    public static Object[][] getCartData() throws Exception {
        return readExcel(ExcelConstants.CART_DATA_PATH, ExcelConstants.CART_SHEET);
    }
}
```

BaseTest.java

```java
package com.company.project.tests;

import com.company.project.drivers.DriverFactory;
import com.company.project.drivers.DriverManager;
import com.company.project.utils.ConfigReader;
import org.openqa.selenium.WebDriver;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;

public class BaseTest {

    @BeforeMethod
    public void setUp() {
        DriverFactory.initDriver(ConfigReader.get("browser"));
        DriverManager.getDriver().get(ConfigReader.get("base.url"));
    }

    public WebDriver getDriver() {
        return DriverManager.getDriver();
    }

    @AfterMethod
    public void tearDown() {
        DriverManager.quitDriver();
    }
}
```

Sample Test — LoginTest.java

```java
package com.company.project.tests;

import com.company.project.pages.LoginPage;
import com.company.project.utils.ExcelUtils;
import io.qameta.allure.*;
import org.testng.Assert;
import org.testng.annotations.Test;

@Epic("Myntra App")
@Feature("Authentication")
public class LoginTest extends BaseTest {

    @Test(groups = {"smoke"},
          dataProvider = "loginData",
          dataProviderClass = ExcelUtils.class)
    @Severity(SeverityLevel.CRITICAL)
    @Story("Login")
    @Description("Verify login with credentials from Excel")
    public void verifyLogin(String username, String password) {
        LoginPage loginPage = new LoginPage(getDriver());
        enterCredentials(loginPage, username, password);
        submitLogin(loginPage);
        verifyDashboard(loginPage);
    }

    @Step("Enter username: {username} and password")
    public void enterCredentials(LoginPage page, String username, String password) {
        page.enterEmail(username);
        page.enterPassword(password);
    }

    @Step("Click login button")
    public void submitLogin(LoginPage page) {
        page.clickLogin();
    }

    @Step("Verify homepage is displayed")
    public void verifyDashboard(LoginPage page) {
        Assert.assertTrue(page.isHomePageDisplayed(), "Login failed");
    }
}
```

testng.xml

```xml
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
<suite name="MyntraSuite" parallel="methods" thread-count="3">

  <listeners>
    <listener class-name="com.company.project.listeners.TestListener"/>
    <listener class-name="com.company.project.listeners.RetryListener"/>
  </listeners>

  <test name="MyntraTests">
    <classes>
      <class name="com.company.project.tests.LoginTest"/>
      <class name="com.company.project.tests.SearchTest"/>
      <class name="com.company.project.tests.ProductTest"/>
      <class name="com.company.project.tests.CartTest"/>
    </classes>
  </test>

</suite>
```

---

smoke-suite.xml

```xml
<suite name="SmokeSuite" parallel="methods" thread-count="3">
  <test name="SmokeTests">
    <groups>
      <run>
        <include name="smoke"/>
      </run>
    </groups>
    <classes>
      <class name="com.company.project.tests.LoginTest"/>
      <class name="com.company.project.tests.SearchTest"/>
      <class name="com.company.project.tests.CartTest"/>
    </classes>
  </test>
</suite>
```

.gitignore

```
target/
*.class
allure-results/
allure-report/
reports/screenshots/*.png
reports/logs/*.log
~$*.xlsx
.idea/
*.iml
.vscode/
.DS_Store
Thumbs.db
```

How to Run

```bash
# Run smoke tests
mvn test -DsuiteXmlFile=src/test/resources/suites/smoke-suite.xml

# Run regression tests
mvn test -DsuiteXmlFile=src/test/resources/suites/regression-suite.xml

# Run by group directly
mvn test -Dgroups=smoke

# Generate Allure report after run
allure serve allure-results
```

Listener Responsibilities

File	Job
`TestListener`	Watches events — screenshot on fail, attach to Allure
`RetryAnalyzer`	Decides retry yes/no — reads max count from config
`RetryListener`	Auto applies RetryAnalyzer to every `@Test`

Key Design Decisions

- **Open/quit browser every test** — full isolation, parallel safe
- **ThreadLocal WebDriver** — no shared state between threads
- **ExcelUtils holds DataProviders** — no separate dataproviders folder
- **ScreenshotUtils merged into TestListener** — one less file
- **@Step in tests** — self documenting, no separate AllureUtils needed
- **Constants folder added later** — AppConstants + ExcelConstants